

Topic 1: Programming in 'C' — Language Fundamentals & Overview

1. High-Yield Conceptual Theory

1.1 Data Types, Representation & Architecture Widths

In standard C (C89/C99/C11 standards tested in GATE), data types have machine-dependent representations, but standard lower bounds are guaranteed:

Type	Typical Size (32-bit / 64-bit LP64)	Minimum Range (Signed)	Minimum Range (Unsigned)	Format Specifier
char	1 byte (8 bits)	−128 to 127 (if signed)	0 to 255	%c
short	2 bytes (16 bits)	−32,768 to 32,767	0 to 65,535	%hd / %hu
int	4 bytes (32 bits)	-2^{31} to $2^{31} - 1$	0 to $2^{32} - 1$	%d / %u
long	4 / 8 bytes	-2^{31} to $2^{31} - 1$ (min 32 bits)	0 to $2^{32} - 1$	%ld / %lu
long long	8 bytes (64 bits)	-2^{63} to $2^{63} - 1$	0 to $2^{64} - 1$	%lld / %llu
float	4 bytes (IEEE 754 single precision)	$\approx \pm 3.4 \times 10^{38}$ (6 decimal digits)	—	%f
double	8 bytes (IEEE 754 double precision)	$\approx \pm 1.7 \times 10^{308}$ (15 decimal digits)	—	%lf

Two's Complement Range

The asymmetry of the two's complement range $[-2^{n-1}, 2^{n-1} - 1]$ stems directly from the positional weight equation for an n -bit binary string $(b_{n-1}b_{n-2} \dots b_0)$:

$$X = -b_{n-1}2^{n-1} + \sum_{i=0}^{n-2} b_i2^i$$

Because the most significant bit (MSB) carries a negative weight (-2^{n-1}) and all other bits carry positive weights, zero $(00 \dots 0_2)$ is grouped with the non-negative numbers. This leaves $2^{n-1} - 1$ positive values, but 2^{n-1} negative values.

Bit Width (n)	Unsigned Range $[0, 2^n - 1]$	Signed Range $[-2^{n-1}, 2^{n-1} - 1]$	Most Negative (MIN)	Most Positive (MAX)
4-bit	$[0, 15]$	$[-8, +7]$	1000 (−8)	0111 (+7)
8-bit	$[0, 255]$	$[-128, +127]$	1000 0000 (−128)	0111 1111 (+127)
16-bit	$[0, 65535]$	$[-32768, +32767]$	0x8000 (−32,768)	0x7FFF (+32,767)
32-bit	$[0, 2^{32} - 1]$	$[-2^{31}, 2^{31} - 1]$	0x80000000 (−2,147,483,648)	0x7FFFFFFF (+2,147,483,647)

Pitfall 1: The INT_MIN Negation & abs() Trap

Because $|-2^{n-1}| = 2^{n-1}$, the absolute value of the most negative number cannot fit within n signed bits.

```
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>

int main(void) {
    int x = INT_MIN; // -2,147,483,648

    // Undefined Behavior in standard C; on 2's complement hardware, -x wraps to
    INT_MIN
    int neg = -x;
    int absolute = abs(x);

    printf("neg: %d\n", neg);           // Prints: -2147483648
    printf("abs: %d\n", absolute);     // Prints: -2147483648
    return 0;
}
```

- **The Hardware Mechanism:** To negate in two's complement, you invert the bits and add 1:
$$\text{For 8-bit } -128 = 1000\,0000_2 \implies \text{Invert: } 0111\,1111_2 \xrightarrow{+1} 1000\,0000_2 \text{ (−128)}$$
- The value is its own additive inverse.
- **GATE/Security Implication:** Code validating input via `if (abs(x) < limit)` will pass a malicious `INT_MIN` because `abs(INT_MIN)` evaluates to a negative value.

Pitfall 2: Carry Flag (C) vs. Overflow Flag (V)

In ALU design and assembly, unsigned and signed overflow are tracked by completely separate hardware flags:

- **Carry Flag (C_{out}):** Indicates whether an arithmetic operation produced an unsigned wrap-around beyond $2^n - 1$.
- **Overflow Flag (V):** Indicates whether signed arithmetic crossed the $[-2^{n-1}, 2^{n-1} - 1]$ boundary, producing a mathematically incorrect sign.

$$V = C_{in} \oplus C_{out} \quad (\text{Carry into MSB} \oplus \text{Carry out of MSB})$$

Alternatively:

$$V = (A_{n-1} \wedge B_{n-1} \wedge \overline{S_{n-1}}) \vee (\overline{A_{n-1}} \wedge \overline{B_{n-1}} \wedge S_{n-1})$$

Case A: 64 + 65 in 8-bit signed

```

0100 0000  (64)
+ 0100 0001  (65)
-----
1000 0001  (-127 in signed, 129 in unsigned)
```

Carry into MSB = 1, Carry out of MSB = 0

Flags: C = 0, V = 1 --> Signed Overflow! (Positive + Positive = Negative)

Case B: -1 + (-1) in 8-bit signed

```

1111 1111  (-1)
+ 1111 1111  (-1)
-----
1 1111 1110  (-2 in signed, 254 in unsigned)
```

Carry into MSB = 1, Carry out of MSB = 1

Flags: C = 1, V = 0 --> Unsigned Wrapped, but Signed result (-2) is valid!

Pitfall 3: Off-by-One Loop Bounds & Wrap-around

Loops relying on signed/unsigned boundary conditions will run indefinitely if the counter wraps past the boundary instead of terminating.

```

#include <stdio.h>
#include <stdint.h>

void signed_loop(void) {
    // 8-bit signed range: [-128, 127]
    // When i == 127, i++ wraps around to -128.
    // -128 <= 127 remains TRUE forever!
    for (int8_t i = 0; i <= 127; i++) {
        // Infinite loop
        break;
    }
}

void unsigned_loop(void) {
```

```

// 8-bit unsigned range: [0, 255]
// When i == 0, i-- wraps around to 255.
// 255 >= 0 remains TRUE forever!
for (uint8_t i = 10; i >= 0; i--) {
    // Infinite loop
    break;
}
}

```

Pitfall 4: Arithmetic vs. Logical Right Shift (>>)

Shifting a signed negative number preserves sign extension (arithmetic shift), which produces surprising boundary states:

```

#include <stdio.h>

int main(void) {
    int x = -1; // All 1s in binary (0xFFFFFFFF)

    // Arithmetic shift propagates the sign bit (1s) from the left
    x = x >> 1;
    printf("%d\n", x); // Still prints -1!

    // Shift loop that never terminates:
    int val = -8;
    while (val > 0) { // val is negative, loop never enters
        val >>= 1;
    }
    while (val != 0) { // Infinite loop: val shifts from -8 -> -4 -> -2 -> -1 -> -1
        ...
        break;
    }
    return 0;
}

```

- For unsigned integers, >> is always **logical** (fills high bits with 0).
- For signed negative integers, C leaves the shift behavior implementation-defined, but nearly all modern hardware and compilers perform an **arithmetic shift** (fills high bits with 1). -1 >> n will never become 0.

Pitfall 5: Hardware Fault on INT_MIN / -1 and INT_MIN % -1

While integer division by zero is a well-known crash, dividing INT_MIN by -1 triggers the exact same hardware CPU exception (#DE - Divide Error / SIGFPE).

```

#include <stdio.h>
#include <limits.h>

int main(void) {
    int a = INT_MIN;
    int b = -1;

    // Mathematical result: +2,147,483,648
    // Cannot fit into signed 32-bit register [-2147483648, 2147483647]
    int c = a / b; // Crashes with Floating Point Exception (Core Dumped)
    int d = a % b; // Also triggers #DE exception

    printf("%d %d\n", c, d);
    return 0;
}

```

- **Under the Hood (x86 assembly):** The `idiv` instruction divides the 64-bit pair `edx:eax` by the operand. When the quotient exceeds the capacity of the 32-bit destination register `eax`, the CPU raises an interrupt trap.

1.2 Implicit Conversions & Integer Promotion Rules

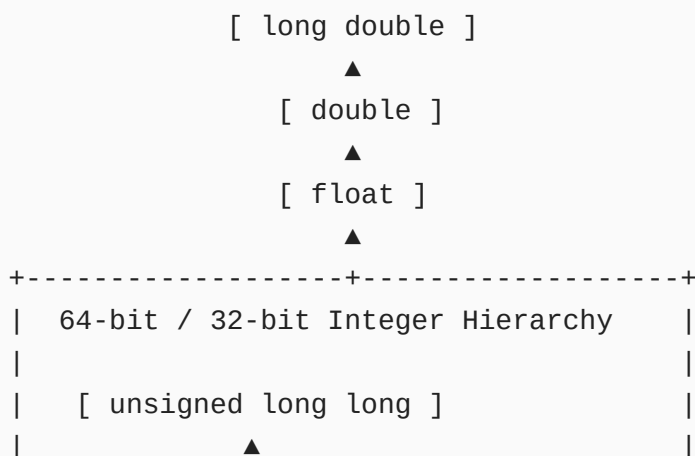
Implicit type conversions in C occur during **Integer Promotion** (applied to types narrower than `int`) and the **Usual Arithmetic Conversions** (applied to bring binary operands to a common type).

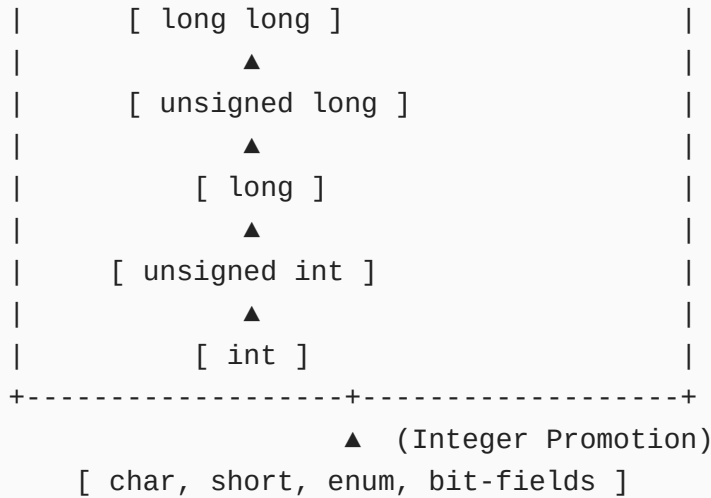
Below is the standard rank hierarchy followed by every major pitfall, step-by-step mechanical evaluations, and prevention strategies.

The Conversion Hierarchy

When two operands of differing types meet in an arithmetic, relational, or bitwise expression:

long double > double > float > unsigned long long > long long > unsigned long > long > unsigned int > int





1. **Integer Promotion First:** Operands smaller than `int` (`char`, signed/unsigned `short`, `enum`, bit-fields) are automatically widened to `int` (or `unsigned int` if `int` cannot represent the full range).
2. **Floating-point Rule:** If either operand is floating-point, the other is converted to the highest floating-point rank present.
3. **Integer Balancing Rule:**
 - If both have the same signedness: the lower rank converts to the higher rank.
 - If the unsigned operand has a rank $\geq$ the signed operand: the signed operand converts to **unsigned**.
 - If the signed operand's type can represent all values of the unsigned operand's type: the unsigned operand converts to the **signed** type.
 - Otherwise: both convert to the **unsigned** version of the signed operand's type.

Pitfall 1: Signed vs. Unsigned Relational Comparison

Comparing a negative signed integer to an unsigned integer forces the signed value to be reinterpreted as an unsigned value modulo 2^N .

```

#include <stdio.h>

int main(void) {
    int a = -5;
    unsigned int b = 3;

    if (a < b) {
        printf("True\n");
    } else {
        printf("False\n"); // Output: False
    }
    return 0;
}

```

- **Step 1:** Operand types are `int` and `unsigned int`.

- **Step 2:** By the Usual Arithmetic Conversions, `int` converts to `unsigned int`.
- **Step 3:** `-5` converted to a 32-bit `unsigned int` becomes:

$$-5 \pmod{2^{32}} = 2^{32} - 5 = 4,294,967,291$$

- **Step 4:** The comparison evaluates $4,294,967,291 < 3$, which is **false**.

Pitfall 2: `sizeof` Operates as `size_t` (Unsigned)

The `sizeof` operator yields type `size_t`, which the C standard defines as an unsigned integer type (typically `unsigned long` or `unsigned long long`).

```
#include <stdio.h>

int main(void) {
    int array[] = {10, 20, 30};
    int i = -1;

    // Trap A: Direct comparison
    if (i < sizeof(array) / sizeof(array[0])) {
        printf("Valid index\n");
    } else {
        printf("Out of bounds\n"); // Output: Out of bounds
    }

    // Trap B: Loop underflow
    // If n = 0, (n - 1) wraps around to SIZE_MAX
    size_t n = 0;
    for (size_t j = 0; j <= n - 1; j++) {
        // Runs 18,446,744,073,709,551,615 times on 64-bit systems!
        break;
    }
    return 0;
}
```

- **Why Trap A fails:** `sizeof(...)` yields `size_t`. Comparing signed `int i = -1` against `size_t` promotes `i` to `(size_t)-1` (`SIZE_MAX`), turning the condition into `18446744073709551615ULL < 3ULL`.
- **Fix:** Cast `sizeof` to `int` if indices can be negative, or enforce `i >= 0 && (size_t)i < count`.

Pitfall 3: Small Unsigned Types Promote to SIGNED `int`

A common misconception is that unsigned types remain unsigned during arithmetic. If `int` can fit all values of the smaller type, C promotes to **signed** `int`.

```
#include <stdio.h>
```

```

int main(void) {
    unsigned short a = 50000;
    unsigned short b = 50000;

    // Both a and b promote to SIGNED int on 32-bit / 64-bit platforms
    // 50000 * 50000 = 2,500,000,000
    // Maximum signed 32-bit int is 2,147,483,647 -> SIGNED OVERFLOW (Undefined
    Behavior)!
    long long c = a * b;

    printf("Result: %lld\n", c); // Typically prints negative garbage due to signed
    UB
    return 0;
}

```

- **Why it happens:** On platforms where `sizeof(short) == 2` and `sizeof(int) == 4`, `int` can hold $[-2^{31}, 2^{31} - 1]$, which fully encompasses `unsigned short` (`[0, 65535]`). Thus, both operands promote to `signed int`, not `unsigned int`.
- **Fix:** Explicitly cast before arithmetic: `(unsigned int)a * b` or `(long long)a * b`.

Pitfall 4: Bitwise Inversion `~` and Sign Extension

Bitwise operations promote narrow types to `int` before inverting, introducing leading `1`s that fail subsequent equality checks.

```

#include <stdio.h>
#include <stdint.h>

int main(void) {
    uint8_t x = 0xAA; // Binary: 10101010

    // Expectation: ~x should be 0x55 (01010101)
    if (~x == 0x55) {
        printf("Matches\n");
    } else {
        printf("Does NOT match\n"); // Output: Does NOT match
    }

    printf("Value of ~x: 0x%X\n", ~x); // Prints: 0xFFFFF55
    return 0;
}

```

- **Execution Trace:**

1. `x` is promoted to `signed int`: `0x000000AA`.

2. The bitwise NOT `~` inverts all 32 bits: `0xFFFFFFFF55` (decimal `-171`).
 3. `0x55` is an `int` literal: `0x00000055`.
 4. `0xFFFFFFFF55 == 0x00000055` evaluates to **false**.
- **Fix:** Mask off promoted bits or cast back: `(uint8_t)~x == 0x55` or `(~x & 0xFF) == 0x55`.

Pitfall 5: Implementation-Defined Signedness of `char`

The C standard allows compilers to treat `char` as signed or unsigned by default. On x86/x64 systems it is typically signed; on ARM it is often unsigned.

```
#include <stdio.h>

int main(void) {
    char c = 0xFF;

    // On x86 (signed char):
    // 0xFF promotes to signed int with sign extension: -1 (0xFFFFFFFF)
    // 0xFF literal is signed int: 255 (0x000000FF)
    if (c == 0xFF) {
        printf("Equal\n");
    } else {
        printf("Not Equal\n"); // Prints "Not Equal" on x86, "Equal" on ARM
    }
    return 0;
}
```

- **Fix:** Use `<stdint.h>` types (`uint8_t` or `int8_t`) instead of bare `char` whenever handling binary data or numeric values.

Pitfall 6: Unary Negation on Unsigned Types

Applying `-` to an unsigned operand does **not** make it negative; it computes the two's complement modulo 2^N .

```
#include <stdio.h>

int main(void) {
    unsigned int u = 10;

    // -u remains unsigned int!
    // Evaluates to: 2^32 - 10 = 4,294,967,286
    if (-u < 0) {
        printf("Negative\n");
    } else {
        printf("Non-negative: %u\n", -u); // Output: Non-negative: 4294967286
    }
}
```

```
    return 0;
}
```

Pitfall 7: The Ternary Operator (?:) Common Type Trap

The second and third operands of a ternary conditional are balanced using the Usual Arithmetic Conversions, regardless of which branch is taken.

```
#include <stdio.h>

int main(void) {
    int condition = 1;
    int negative = -50;
    unsigned int positive = 100;

    // Both branches convert to unsigned int
    // Even though condition == 1 selects 'negative', it evaluates as unsigned int!
    printf("%u\n", condition ? negative : positive); // Output: 4294967246
    return 0;
}
```

Pitfall 8: Floating-Point Rank and Precision Truncation

When mixing large integer types (unsigned long long or 32-bit int) with single-precision float , the integer is implicitly converted to float . Single-precision IEEE 754 floats only have 24 bits of mantissa precision.

```
#include <stdio.h>

int main(void) {
    int large_val = 16777217; // 2^24 + 1 (requires 25 bits of precision)
    float f = 0.0f;

    // large_val implicitly converts to float, losing precision
    if (large_val == (large_val + f)) {
        printf("Equal!\n"); // Output: Equal!
    } else {
        printf("Not Equal!\n");
    }
    return 0;
}
```

- **Step-by-step:** In `large_val + f` , `large_val` converts to `float` . 16,777,217 cannot be represented exactly in standard 32-bit float and rounds to 16,777,216.0f. When compared, both sides become

`float` , masking the error.

1.3 Operator Precedence & Associativity

Operators are evaluated based on precedence (higher binds tighter) and associativity (direction when operators have identical precedence).

Level	Operators	Description	Associativity
1	<code>() [] -> . ++ (postfix) -- (postfix)</code>	Function call, subscript, member access, postfix	Left to Right
2	<code>++ (prefix) -- (prefix) + - (unary) ! ~ * (deref) & (address) sizeof (type)</code>	Unary operators, type cast	Right to Left
3	<code>* / %</code>	Multiplicative	Left to Right
4	<code>+ -</code>	Additive	Left to Right
5	<code><< >></code>	Bitwise Shift	Left to Right
6	<code>< <= > >=</code>	Relational	Left to Right
7	<code>== !=</code>	Equality	Left to Right
8	<code>&</code>	Bitwise AND	Left to Right
9	<code>^</code>	Bitwise XOR	Left to Right
10	<code>\ </code>	Bitwise OR	Left to Right
11	<code>&&</code>	Logical AND (Short-circuit)	Left to Right
12	<code>\ \ </code>	Logical OR (Short-circuit)	Left to Right
13	<code>?:</code>	Ternary conditional	Right to Left
14	<code>= += -= *= /= %= &= ^= \ = <=> >>=</code>	Assignment operators	Right to Left
15	<code>,</code>	Comma operator	Left to Right

1.4 Sequence Points and Order of Evaluation

- **Associativity and Precedence $\neq$ Order of Evaluation:**

In `f() + g() * h()` , `g() * h()` is computed before adding to `f()` , but whether `f()` , `g()` , or `h()` is called first is **unspecified**.

- **Sequence Point Definition:** A point in execution where all side effects of previous evaluations are guaranteed to be complete, and no side effects of subsequent evaluations have taken place.
- Major sequence points:
 1. End of a full statement (at the semicolon `;`).

2. The boolean short-circuit operators: `&&` , `||` .
 3. The conditional operator `? :` after the condition expression.
 4. The comma operator `,` (e.g., `expr1 , expr2` evaluates `expr1` , discards it, then evaluates `expr2`).
 5. Before a function call (after all arguments are evaluated).
- **Undefined Behavior Warning:** Modifying a scalar object more than once between two consecutive sequence points, or reading its value other than to determine the value to be stored, is **Undefined Behavior**:
 - `i = i++ + 1;` → **UB**
 - `printf("%d %d", ++i, i++);` → **UB**
 - `a[i] = i++;` → **UB**
-

1.5 Short-Circuit Evaluation

- `A && B` : `B` is evaluated **if and only if** `A` evaluates to non-zero (true).
 - `A || B` : `B` is evaluated **if and only if** `A` evaluates to zero (false).
 - Any expressions inside the skipped operand that have side effects (e.g., `++x` , function calls, assignments) are **completely ignored**.
-

1.6 Storage Classes & Linkage

Storage Class	Storage Location	Default Initial Value	Scope	Lifetime	Linkage
<code>auto</code>	Stack	Garbage	Block local	Block duration	None
<code>register</code>	CPU Register / Stack	Garbage	Block local	Block duration	None
<code>static</code> (local)	Data segment (BSS/Data)	Zero (<code>0</code>)	Block local	Program execution	None
<code>static</code> (global)	Data segment (BSS/Data)	Zero (<code>0</code>)	File	Program execution	Internal
<code>extern</code>	Data segment	Zero (<code>0</code>)	Global	Program execution	External

- **Important:** You cannot take the address of a variable declared with `register` (e.g., `®_var` is a compile-time error).
- Local `static` variables retain their value across multiple function invocations and are initialized **only once** at load time.

2. Common GATE Pitfalls & Edge Cases

1. The Comma Operator vs. Comma Separator:

- In `int a = (1, 2, 3);`, `,` is the comma operator: evaluates left-to-right and yields `3`.
- In `int a = 1, 2, 3;`, it fails compilation because `,` acts as a declarator separator without parentheses.
- In `printf("%d", (x, y));`, `y` is passed. In `func(a, b);`, `,` is a parameter separator, and the order of argument evaluation is unspecified.

2. Shift Operator Traps:

- Shifting by a negative count or by $\geq$ the bit-width of the promoted operand is **Undefined Behavior** (e.g., for 32-bit `int x`, `x << 32` or `x >> -1` is UB).
- For signed integers, right-shift (`>>`) of a negative value is implementation-defined (usually arithmetic shift, preserving the sign bit). Right-shift of an unsigned integer is always a logical shift (zero-filled).

3. `sizeof` is a Compile-Time Operator:

- `sizeof(expr)` does not evaluate `expr` at runtime (except for C99 variable-length arrays).
- Example:

```
int i = 5;
sizeof(++i);
printf("%d", i); // Outputs 5, NOT 6!
```

4. Bitwise vs. Logical Inversion:

- `~0` : bitwise NOT on `0` (all 32 bits become 1s, representing `-1` in two's complement).
- `!0` : logical NOT on `0`, yielding integer `1`.

3. Previous Year GATE Questions with Step-by-Step Solutions

PYQ 1 (GATE CS 2019)

Question:

Consider the following C program:

```
#include <stdio.h>
int main() {
    int x = 1;
    if (x > 0)
```

```

    x++;
    if (x > 1)
        x += 2;
    if (x > 2)
        x += 3;
    printf("%d", x);
    return 0;
}

```

What is the output printed by the program?

Step-by-step Solution:

1. Initial state: $x = 1$.
2. First condition: `if (x > 0)` . Since $1 > 0$ is true, `x++` executes. Now $x = 2$.
3. Second condition: `if (x > 1)` . Since $2 > 1$ is true, `x += 2` executes. Now $x = 2 + 2 = 4$.
4. Third condition: `if (x > 2)` . Since $4 > 2$ is true, `x += 3` executes. Now $x = 4 + 3 = 7$.
5. Final printed value: 7 .

PYQ 2 (GATE CS 2014)

Question:

Consider the following C function:

```

int fun(int n) {
    int x = 1, k;
    if (n == 1) return x;
    for (k = 1; k < n; ++k)
        x = x + fun(k) * fun(n - k);
    return x;
}

```

What is the return value of `fun(5)` ?

Step-by-step Solution:

Let $f(n)$ denote `fun(n)` .

- Base case: $f(1) = 1$.
- For $n > 1$:

$$f(n) = 1 + \sum_{k=1}^{n-1} f(k) \cdot f(n-k)$$

Compute iteratively:

- $n = 1$: $f(1) = 1$
- $n = 2$:

$$f(2) = 1 + f(1) \cdot f(1) = 1 + 1 \cdot 1 = 2$$

- $n = 3$:

$$f(3) = 1 + [f(1)f(2) + f(2)f(1)] = 1 + [1 \cdot 2 + 2 \cdot 1] = 1 + 4 = 5$$

- $n = 4$:

$$f(4) = 1 + [f(1)f(3) + f(2)f(2) + f(3)f(1)] = 1 + [1 \cdot 5 + 2 \cdot 2 + 5 \cdot 1] = 1 + [5 + 4 + 5] = 15$$

- $n = 5$:

$$\begin{aligned} f(5) &= 1 + [f(1)f(4) + f(2)f(3) + f(3)f(2) + f(4)f(1)] \\ &= 1 + [1 \cdot 15 + 2 \cdot 5 + 5 \cdot 2 + 15 \cdot 1] \\ &= 1 + [15 + 10 + 10 + 15] = 1 + 50 = 51 \end{aligned}$$

Output: 51

PYQ 3 (GATE CS 2004)

Question:

Consider the following snippet:

```
int a = 10, b = 20, c = 30;
if (c > b > a)
    printf("TRUE");
else
    printf("FALSE");
```

What is printed?

Step-by-step Solution:

1. Look at operator precedence and associativity: `>` has left-to-right associativity.
2. `c > b > a` is grouped as: `(c > b) > a`.
3. Evaluate `c > b`: `30 > 20` is true, producing integer `1`.
4. Now evaluate: `1 > a` $\rightarrow$ `1 > 10`, which is false (`0`).
5. The `else` branch triggers.

Output: FALSE

4. Tricky Practice Questions (Edge Cases & Traps)

Practice Question 1: Short-Circuiting with Side Effects

Question:

What will be the output of the following C program?

```
#include <stdio.h>
int main() {
    int a = 0, b = 2, c = 3;
    int res = ++a || ++b && ++c;
    printf("%d %d %d %d", a, b, c, res);
    return 0;
}
```

Detailed Explanation:

- Expression: `++a || ++b && ++c`
- Operator precedence: `&&` has higher precedence than `||`.
- Grouping conceptually: `(++a) || ((++b) && (++c))`
- Execution rule:** Even though `&&` has higher precedence, `||` must evaluate its left operand first because `||` introduces a sequence point.
- Left operand: `++a`
 - `a` becomes $0 + 1 = 1$.
 - Since $1 \neq 0$ (true), the condition for `||` is already satisfied.
- Short-circuiting takes effect:** The entire right operand `((++b) && (++c))` is **skipped**.
- Therefore, `b` remains `2`, `c` remains `3`.
- `res` is assigned `1` (result of logical OR).

Output: `1 2 3 1`

Practice Question 2: `sizeof` Side-Effect Invariance

Question:

What is the output of the following code snippet on a standard 32/64-bit architecture?

```
#include <stdio.h>
int main() {
    int x = 10;
    int y = sizeof(x += 5);
}
```

```
printf("x = %d, y = %d", x, y);
return 0;
}
```

Detailed Explanation:

1. The operand of `sizeof` is `x += 5`.
2. `sizeof` determines the size of the *type* that would result from evaluating the expression. The expression `x += 5` yields an `int`.
3. On standard systems, `sizeof(int) == 4`.
4. Expressions inside `sizeof` are **unevaluated operands** at runtime; compiler discards execution code.
5. `x += 5` never executes. Thus, `x` remains `10`.

Output: `x = 10, y = 4`

Practice Question 3: Sign Extension and Integer Promotion

Question:

Consider the following program:

```
#include <stdio.h>
int main() {
    char c = 0xFF;
    if (c == 0xFF)
        printf("Equal\n");
    else
        printf("Not Equal\n");
    return 0;
}
```

Assuming `char` is signed by default (standard two's complement 8-bit), what will be printed?

Detailed Explanation:

1. `c` is initialized with `0xFF` (11111111_2). In an 8-bit signed representation, `0xFF` represents `-1`.
2. When comparing `c == 0xFF`:
 - `c` is of type `char` and undergoes **integer promotion** to signed `int`.
 - Sign extension preserves the negative sign: `-1` (8-bit) becomes `-1` (32-bit: `0xFFFFFFFF`).
 - The integer constant `0xFF` is positive 255 (`0x000000FF`).
3. Now the comparison is: `-1 == 255`, which evaluates to false (0).

Output: Not Equal

5. Quick Revision & Edge-Case Checklist

- ☐ `x == y == z` does **not** check if all three are equal. It evaluates as `(x == y) == z`.
 - ☐ Unsigned and signed comparison: signed integer gets cast to unsigned. Negative numbers become large positive values.
 - ☐ In switch-case statements, missing `break` causes unintentional fall-through across labels.
 - ☐ Relational chains like `a < b < c` evaluate as `(a < b) < c`, comparing boolean `{0, 1}` with `c`.
 - ☐ `sizeof('a')` in C evaluates to `sizeof(int)` which is **4** (character literals in C have type `int`), whereas in C++ it evaluates to **1**.
 - ☐ Storage class `register` forbids the `&` (address-of) operator.
 - ☐ Static variables are stored in the data/BSS segment and zero-initialized automatically.
-